
Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #1

SCADE One Fundamentals

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: June 29, 2026

Student Submission

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Install and verify the toolchain:** install SCADE One and confirm that the KCG code generator runs without errors, since it is the central tool of every subsequent practice.
- ✓ **Reason about synchronous dataflow:** explain how a SCADE operator is evaluated exactly once per execution cycle and why this produces deterministic behavior.
- ✓ **Build stateful logic:** implement a self-resetting 0–20 counter using the `pre` operator and modular arithmetic.
- ✓ **Build and validate combinational logic:** implement a piecewise saturation function with chained `if-then-else` and verify both models in the SCADE One simulator before any code generation.

2 Introduction

In certified avionics, critical algorithms are rarely hand-coded in C; they are first modeled graphically, validated by simulation, and only then translated into certifiable code by a qualified generator. SCADE One implements exactly this workflow and is the foundation on which the landing gear control system is built throughout the course. This first session establishes the discipline that the rest of the series depends on: construct a model, simulate it, and trust the simulation before touching hardware. Because no microcontroller is involved yet, the focus stays entirely on the synchronous dataflow semantics that make a SCADE model deterministic and analyzable.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (An-sys)	1	Licensed or student version; KCG enabled
2	ESP32 DevKit V1	1	Not used in this practice (see note)
3	USB-A to Micro-USB cable	1	Not used in this practice (see note)
4	ESP-IDF (v5.x)	1	Installed in Practice 2

Hardware

This practice is software-only and requires **no hardware**. The ESP32 and ESP-IDF toolchain are listed for course continuity and are exercised starting in Practice 2. Only a working SCADE One installation is needed here.

4 Theory & Block Reference

4.1.

In SCADE One every signal and port has a **static type** declared at model time. The type checker enforces compatibility at every connection: wiring an `int32` output to a `bool` input is a compile-time error, never a runtime surprise. Table 1 lists the primitive types used throughout this practice series.

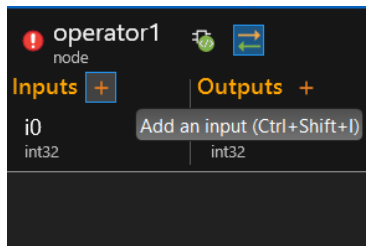
Type	Range / Values	Typical use
<code>bool</code>	<code>true</code> , <code>false</code>	Flags, conditions, enable signals
<code>int8</code>	−128 to 127	Small counters, compact status codes
<code>int16</code>	−32 768 to 32 767	Mid-range integers
<code>int32</code>	-2^{31} to $2^{31} - 1$	General-purpose integers; default choice
<code>float32</code>	IEEE 754 single precision (≈ 7 sig. digits)	Sensor readings, percentages, angles
<code>float64</code>	IEEE 754 double precision (≈ 15 sig. digits)	High-precision computations

Table 1: Primitive types available in SCADE One.

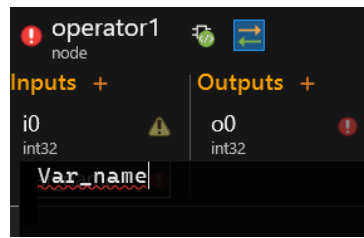
Type selection

Unless a port specification or range constraint forces a narrower type, use `int32` for integers and `float32` for real-valued signals. Mixing signed and unsigned or narrowing without an explicit cast are caught by the type checker before any code is generated.

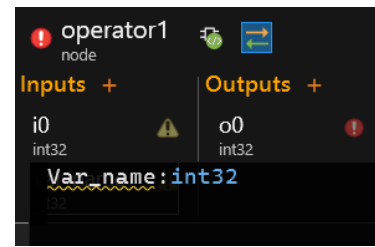
The figures below show how to declare a new input or output variable in the SCADE One operator editor. Repeat the same procedure for every port you add to an operator.



(a) Right-click the operator border and select *Add Input* (or *Add Output*).



(b) Type the variable name; avoid spaces and reserved keywords.



(c) Select the type from the drop-down (e.g. `int32`) and confirm.

Figure 1: Declaring a typed variable in the SCADE One operator editor.

4.2.

A SCADE operator is evaluated once per cycle. Every block reads its inputs, produces its outputs for that cycle, and (unless it carries explicit memory) keeps no internal state. The blocks below are the minimum set required to build this practice.

Block:
+
-
<
>

These four are **combinational** (no memory) and evaluated pointwise every cycle. Operand types must match (`Int` with `Int`, `Float` with `Float`); the relational operators always return a `Bool`.

Op	In	Out	Result
+	2 numeric	numeric	sum of the two operands
-	2 numeric	numeric	left minus right
<	2 numeric	Bool	<code>true</code> iff left < right
>	2 numeric	Bool	<code>true</code> iff left > right

Behavior: stateless; the output depends only on the current-cycle inputs. Mixing incompatible numeric types is rejected by the type checker rather than silently coerced.

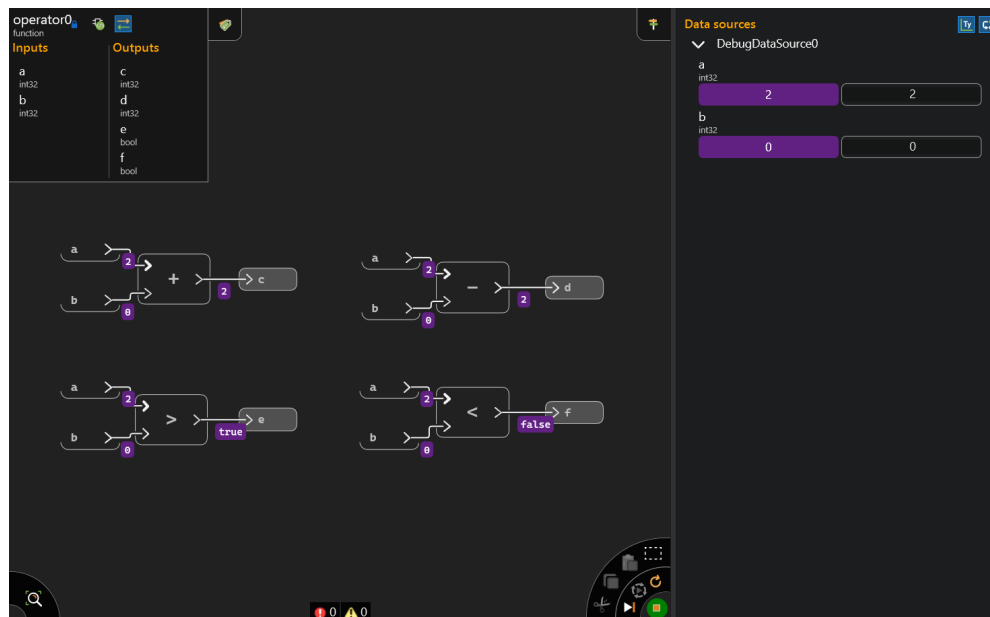


Figure 3: All four operators wired in a single operator ($a = 2$, $b = 0$). The simulation panel on the right shows the values propagated through each block in the current cycle: $c = 2$, $d = 2$, $e = \text{true}$, $f = \text{false}$.

Block: Initialized Unit Delay (pre)



Figure 4: Block Initialized Unit Delay — two-input delay with explicit first-cycle value.

Ports:

- **Initial value (top-left):** type T — the value emitted on the *first* cycle only.
- **Signal (bottom-left):** type T — the stream whose previous value is returned from cycle 2 onwards.
- **Output (right):** type T — **initial** on cycle 1, **pre signal** on every subsequent cycle.

Behavior:

This block is the dataflow equivalent of `init -> pre signal`. It solves the initialization problem in a single node: because `pre` alone is undefined at cycle 1, the top input supplies a safe default so the output is well-defined from the very first execution. Both inputs must share the same type. A feedback loop that routes the output back into the bottom input — combined with a constant on the top input — is the standard pattern for stateful counters and accumulators in SCADE One.

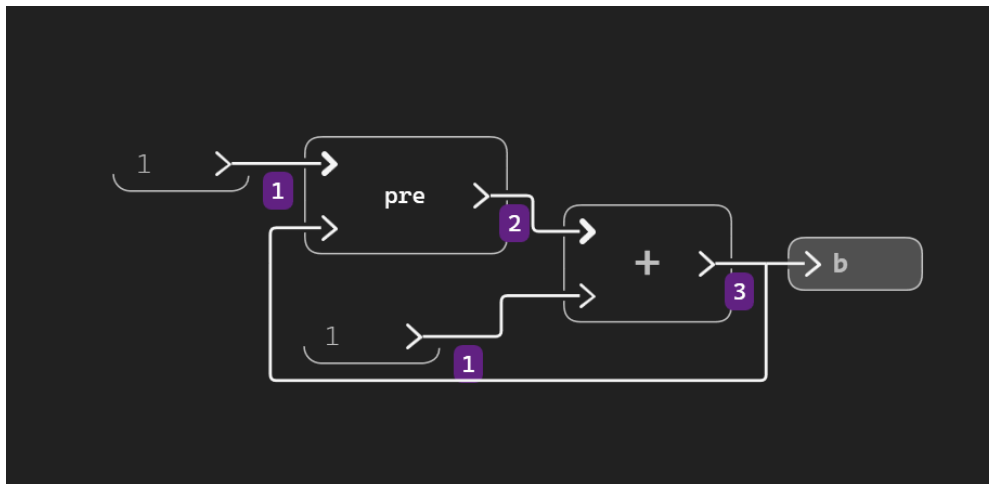


Figure 5: Initialized unit delay used as a counter: the top input supplies the constant 1 as the first-cycle seed; the bottom input closes the feedback loop by routing the current output b back into the block. On cycle 1 the block emits 1; the $+$ node adds another 1 and writes 2 to b . On every subsequent cycle the block emits the previous value of b , producing the sequence 1, 2, 3, ... Each numbered probe (1–3) marks a signal node where the simulator displays the live value.

Block: if-then-else

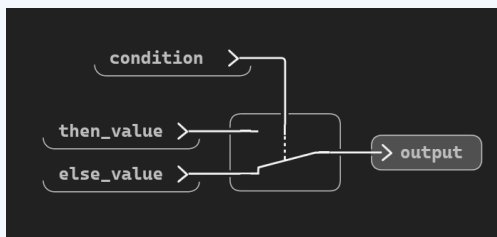


Figure 6: Block if-then-else — conditional selector.

Ports:

- **Condition (top-left):** Bool — selects the branch.
- **Then-value (mid-left):** type T — output when the condition is **true**.
- **Else-value (bottom-left):** type T — output when the condition is **false**.
- **Output (right):** type T — the selected value.

Behavior:

A purely combinational selector: **if c then a else b**. Both branches must have the *same* type, and both are always type-checked even though only one is emitted per cycle. Conditionals chain to express piecewise functions, e.g. **if x < 0 then 0 else (if x > 10 then 10 else x)**.

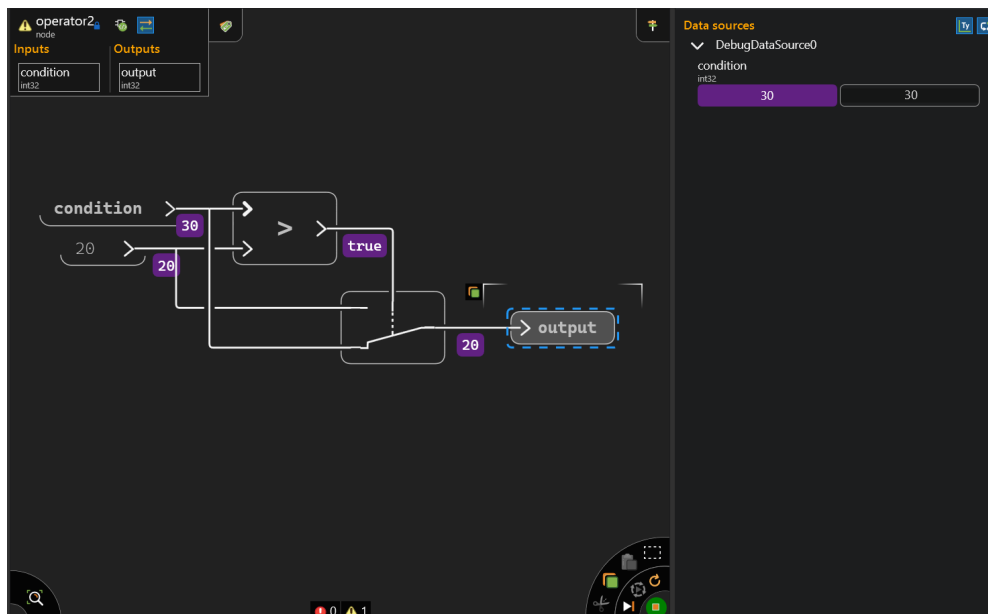


Figure 7: **if-then-else** used as an upper clamp: `condition = 30` is compared against the constant 20 by the `>` block, which evaluates to `true`. The **if-then-else** block therefore selects the then-branch and emits 20 instead of 30, clamping the output to the defined ceiling. Any input value at or below 20 would make the condition `false` and pass through unchanged — making this pattern the building block for range-limiting signals such as sensor readings or command references.

Block: mod

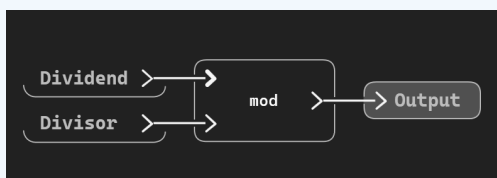


Figure 8: Block **mod** — modular arithmetic.

Ports:

- **Dividend (top-left):** Int — value to reduce.
- **Divisor (bottom-left):** Int — modulus.
- **Output (right):** Int — remainder of the division.

Behavior:

$a \bmod b$ returns the remainder of a / b , an integer in $[0, b-1]$ for positive operands. Combined with **pre** it produces a cyclic counter without any state machine: $(\text{pre count} + 1) \bmod 21$ cycles through 0..20 and wraps automatically. A divisor of 0 is undefined and must never reach the port.

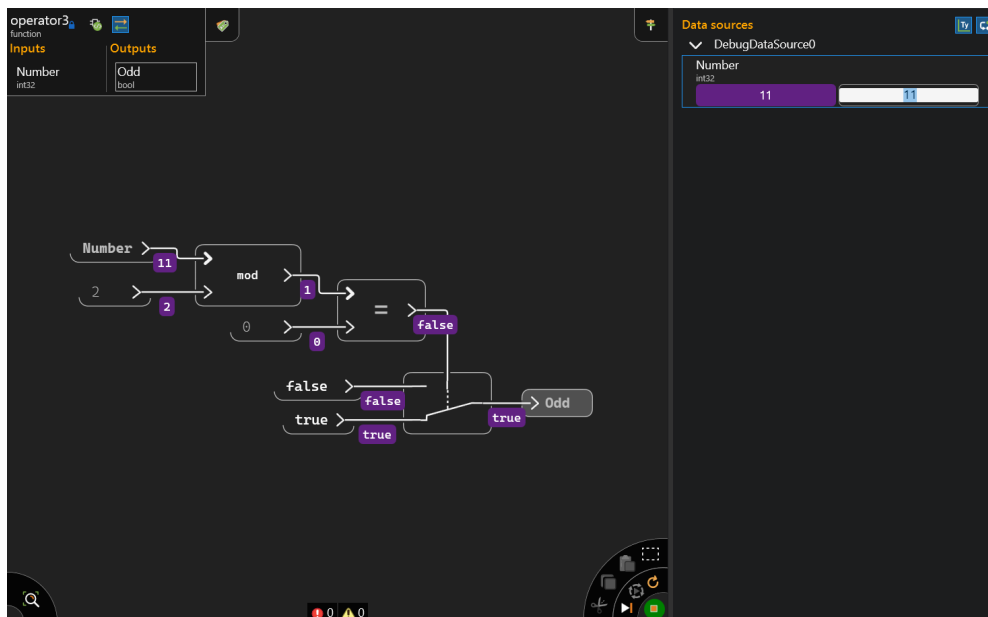


Figure 9: `mod` used as an odd-number detector: `Number = 11` is divided by the constant `2`, yielding remainder `1`. The `=` block compares that remainder against `0`; since $1 \neq 0$ the result is `false`, so the if-then-else block selects the `true` branch and emits `Odd = true`. An even input would produce remainder `0`, flip the condition to `true`, and emit `Odd = false`. This illustrates the general pattern: $x \bmod n = 0$ is the standard dataflow test for divisibility in SCADE One.

5 Worked Example

5.1.

Before building the stateful counter, become comfortable with the editor and the simulator using a small *combinational* operator. You will build **AddCompare**: it takes two integers `a` and `b` and produces their sum and the Boolean result of `a > b`. This uses only a subset of the blocks above (`+` and `>`) and should take 20–30 minutes.

5.2.

1. **Open SCADE One and create a new project.**

File → New → SCADE Suite Project. Name it `Example_1`.

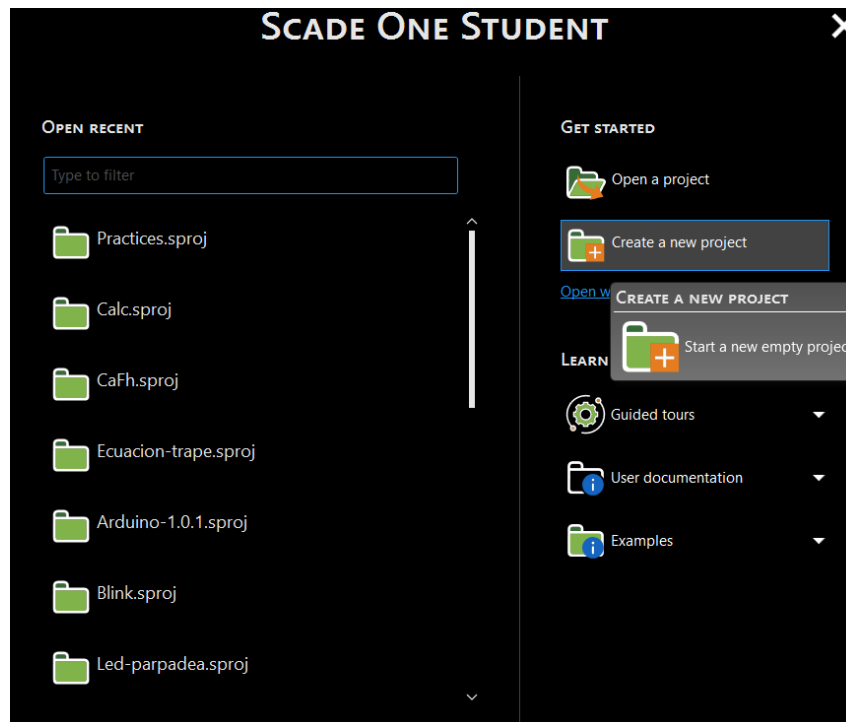
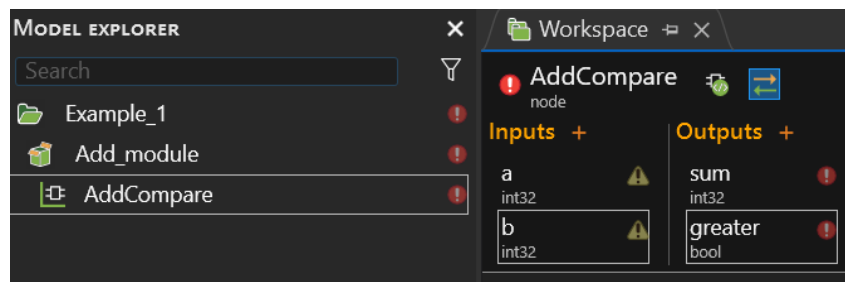


Figure 10: Creating a new project in SCADE One.

2. Create the operator.

Create a new module body named `Add_module`. Then add a new operator named `AddCompare`. Declare two inputs `a : int32` and `b : int32`, and two outputs `sum : int32` and `greater : bool`.

Figure 11: Declaring the inputs and outputs of `AddCompare`.

3. Drop the operator blocks.

From the palette, place a `+` block and a `>` block on the diagram.

4. Wire the dataflow.

Connect `a` and `b` to both blocks. Route the `+` output to `sum` and the `>` output to `greater`.

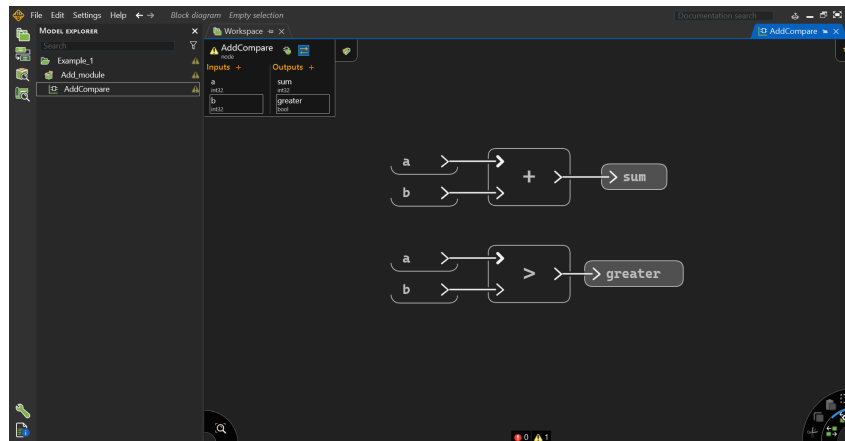


Figure 12: Completed dataflow for AddCompare.

5. Simulate.

Open the simulator, set $a = 7$, $b = 3$, and execute one cycle. Verify $\text{sum} = 10$ and $\text{greater} = \text{true}$. Change $b = 9$ and confirm $\text{greater} = \text{false}$. There is no pre, so each cycle is independent of the last.

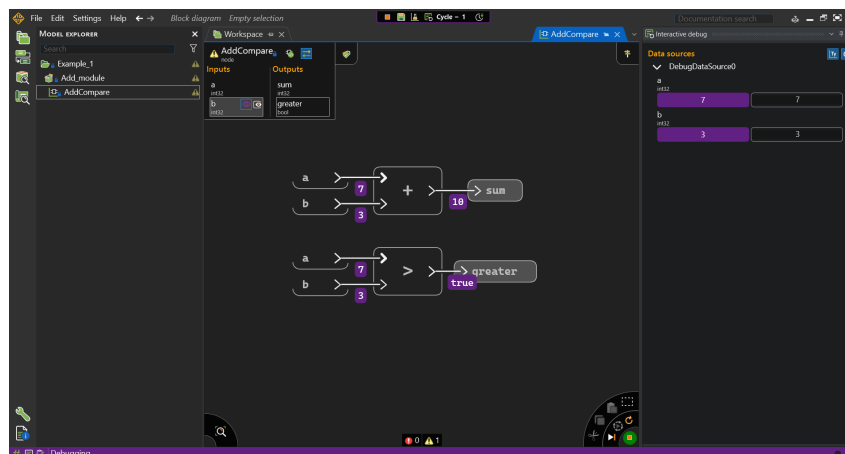


Figure 13: Simulator output for two input combinations.

6 Main Activity

6.1.

Build a single SCADE operator named **BoundedCounter** that counts from 0 to 20, wraps back to 0, and exposes a second output that clamps the count to the range $[0, 10]$. The operator has no inputs; both outputs are driven entirely by internal dataflow. This connects the two core concepts of the practice — stateful accumulation and combinational range limiting — into one coherent model that you will validate end-to-end in the simulator.

6.2.

- **Spec 1:** Output `count : int32` starts at 0 on the first cycle and increments by 1 each cycle, wrapping back to 0 after reaching 20.

- **Spec 2:** Output `saturated : int32` mirrors `count` for values in `[0, 10]` and holds 10 for any count above 10.
- **Spec 3:** No state machine is used; the entire behavior is expressed with the blocks introduced in Section 4.
- **Spec 4:** The model is verified in the simulator across at least one full 0–20 period before any code is generated.

6.3.

1. **Create the project and operator.**

Start a new SCADE Suite Project named `Practice_1` and add an operator named `BoundedCounter` with no inputs and two outputs: `count : int32` and `saturated : int32`.

2. **Implement the cyclic counter.**

Wire the dataflow so that `count` increments by 1 each cycle and resets after 20. You will need a block that introduces memory, a block that handles the arithmetic, and a block that enforces the wrap. Make sure the output is well-defined on the very first cycle.

3. **Implement the saturation output.**

Drive `saturated` from `count` using only comparisons and conditional selection. No additional memory is needed here.

4. **Simulate and verify.**

Open the simulator and step through at least one full period. Confirm that `count` cycles 0, 1, ..., 20, 0, ... and that `saturated` tracks `count` up to 10 and then stays flat. Record the cycle at which the first wrap occurs and the first cycle where the two outputs diverge.

7 Expected Results

Expected Results

- **SCADE Simulation:** `count` produces a sawtooth that increments each cycle and wraps back to 0 at cycle 21. `saturated` is identical to `count` while the count stays at or below 10; from cycle 11 onward it holds 10 while `count` continues rising. Both outputs reset to 0 simultaneously on the wrap and resume tracking each other from 0 again. Both signals should be visible on the simulator's waveform panel simultaneously.
- **Console / UART Output:** This practice is software-only; there is no serial output. The simulator's signal table is the equivalent diagnostic, e.g.:

```
cycle 10 | count = 10 | saturated = 10
cycle 11 | count = 11 | saturated = 10
cycle 21 | count = 0  | saturated = 0
```

```
cycle 22 | count = 1 | saturated = 1
```

UART reporting is introduced in Practice 2.

- **Physical Behavior:** None — no hardware is connected. The deliverable is the `BoundedCounter` operator with a passing simulation trace covering at least one full 0–20 period.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] The `Initialized Unit Delay` block has two inputs. What value does each input contribute on cycle 1, and what value does each input contribute on cycle 2 onwards? Explain what would happen to the output if the top input were left disconnected.

Q2. [*Analysis*] In `BoundedCounter`, `saturated` is driven by a chain of `if-then-else` blocks. What happens to `saturated` if the ceiling constant (10) is changed to a value greater than 20? Trace the output for the full 0–20 period and explain the result.

Q3. [*Analysis*] You have two options for driving `saturated`: (a) chain two `if-then-else` blocks as implemented, or (b) use a single `if-then-else` with the condition `count > 10`. Under what inputs do these produce the same result, and when would they differ?

Q4. [*Synthesis*] Modify `BoundedCounter` to add a third output `parity : bool` that is `true` when `count` is even and `false` when it is odd. Describe which blocks from Section 4 you would use and how you would wire them. No new block types are allowed.

Q5. [*Critical*] Identify what happens to `BoundedCounter` if the feedback wire from the output is connected to the *top* input of the `Initialized Unit Delay` instead of the bottom

input. Would SCADE One detect this at model time or would it compile silently and produce incorrect behavior at runtime? Justify your answer.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to the broader context of model-based design for critical systems.

10 Bonus Challenge

★ Bonus Challenge

Challenge:

Extend `BoundedCounter` into a **triangle-wave generator**: the output must ramp up $0 \rightarrow 40$, then ramp *down* $40 \rightarrow 0$, and repeat this triangular pattern forever. You may **not** use a state machine — build it purely from the dataflow blocks of this practice (`pre`, `if-then-else`, `mod`, `+`, `-`). Validate the full up-and-down period in the simulator.

Hint:

A sawtooth counter that runs from 0 to 79 with `mod 80` already contains one full triangle period. Think about how you can map the range `[0, 79]` onto a value that rises from 0 to 40 and then falls back to 0 — without any explicit direction flag. The `mod` and `if-then-else` blocks are enough to perform that mapping on top of the counter you already built.

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: Causality error on a feedback loop

Symptom: SCADE One reports a causality error on the wire that feeds the output of the operator back into the `pre` block, and the model refuses to compile.

Cause: The `pre` block requires its input to be available from the *previous* cycle, not the current one. If the feedback wire is connected directly without going through the initialized unit delay, SCADE detects a combinational cycle with no memory element to break it and flags the model as causally incorrect.

Fix: Verify that the feedback wire enters the *bottom* (signal) input of the initialized unit delay block and that the *top* input carries the first-cycle constant. A direct wire from output back to a plain arithmetic input with no `pre` in between is never valid in synchronous dataflow.

Issue: Operator defined as function instead of node

Symptom: SCADE One rejects the operator with an error stating that a `function` cannot contain memory elements, even though the diagram looks correct.

Cause: SCADE One distinguishes between `function` (stateless, combinational) and `node` (stateful, may contain `pre`). Declaring the operator as a `function` while using an initialized unit delay inside it violates that contract.

Fix: In the operator properties, change the kind from `function` to `node`. Any operator that contains a `pre` block or calls another `node` must itself be declared as a `node`.